UNIVERSIDAD AUTÓNOMA GABRIEL RENÉ MORENO
FACULTAD DE INGENIERÍA EN CIENCIAS DE LA COMPUTACIÓN Y TELECOMUNICACIONES



INFORME TÉCNICO
Opinómetro de Restaurantes – Análisis de sentimiento de reseñas

INTEGRANTES:

- DURAN MARCELO KELLY ALLISON 221044434
- QUISPE LERO FABIOLA MARIOMA 215054067
- SANCHEZ CUELLAR MELISSA 215164441
- MAMANI VILLCA WENDY ESTHEFANIE 217028799

MATERIA: Tópicos. Avanzada De Programación (Algorit. Gene)
GRUPO: **SB**
DOCENTE: Ing. kenji Kawaida

GESTIÓN: II-2026
SANTA CRUZ DE LA SIERRA - BOLIVIA

1. Problema **3**
1.1 Descripción del problema 3
1.2 Objetivos 3
Objetivo general 3
Objetivos específicos 4
1.3 Alcance y limitaciones 4
2. Dataset **5**
2.1 Origen, licencia y fecha de obtención 5
2.2 Descripción de las columnas 5
2.3 Preparación de los datos 5
2.4 Análisis exploratorio 6
3. Decisiones de NLP **8**
3.1 Normalización 8
3.2 Tokenización 9
3.3 Stop words 10
Decisiones de filtrado de Stop Words 10
Ejemplos de antes y después 12
Lo que muestran los ejemplos: 13
3.4 Stemming vs lematización 13
3.5 N-gramas y TF-IDF 15
4. Modelos **16**
4.1 Modelos elegidos y justificación 16
4.2 Línea base 16
4.3 Tratamiento del desbalance 16
4.4 Ajuste de hiperparámetros 16
4.5 Tabla comparativa de modelos 16
Comparación del rendimiento de los modelos 16
Análisis comparativo de los resultados 17
Conclusión de la comparación 17
4.6 Análisis de errores 18
5. Arquitectura **18**
5.1 Capas del sistema 18
5.1.1. Capa de Presentación (Frontend SPA) 19
5.1.2. Capa de Enrutamiento y Exposición (API REST) 20
5.1.3. Capa de Validación y Esquemas (DTOs) 20
5.1.4. Capa de Servicio y Lógica de Negocio 20
5.1.5. Capa de Machine Learning y NLP (Pipeline scikit-learn) 21
5.2 Diagrama de clases 24
5.3 Casos de uso 25
5.4 Endpoints de la API 27
5.4.1 Catálogo de Endpoints 27
5.4.2 Contratos de Datos y Ejemplos de Petición / Respuesta 28
A. Inferencia individual (POST /api/v1/predict) 28

B. Métricas del Modelo en Producción (GET /api/v1/metrics)..... 28
C. Estado de Salud del Servicio (GET /api/v1/health)..... 29
5.4.3 Reglas de Validación Defensiva (Esquemas Pydantic)..... 30
5.4.4 Suite de Pruebas Automatizadas (pytest)..... 30
5.5 Frontend..... 32
6. Uso de inteligencia artificial..... 34
Sanchez Cuellar Melissa (Datos y NLP)..... 34
Duran Marcelo Kelly Allison (Backend)..... 34
7. Referencias..... 35

1. Problema

1.1 Descripción del problema

Los restaurantes reciben una gran cantidad de reseñas en plataformas como TripAdvisor, donde los clientes cuentan su experiencia con la comida, el servicio, el ambiente y los precios. Estas opiniones son muy valiosas para los dueños, que pueden detectar qué funciona y qué deben mejorar, y para otros clientes, que las usan para decidir a dónde ir.

Sin embargo, leer y clasificar manualmente miles de reseñas es lento y poco práctico. Además, las reseñas están escritas en lenguaje informal: tienen faltas de ortografía, palabras sin tildes, letras repetidas para dar énfasis (“riquísimooo”), signos de puntuación exagerados y negaciones que cambian el sentido de una frase (“no me gustó”). Por eso no basta con buscar palabras como “bueno” o “malo”.

El problema que resuelve este proyecto es clasificar automáticamente el sentimiento de una reseña de restaurante en español como positivo o negativo, a partir de su texto, indicando además qué tan seguro está el modelo de su predicción.

1.2 Objetivos

Objetivo general

Desarrollar una aplicación web que clasifique el sentimiento de reseñas de restaurantes en español como positivo o negativo, integrando un pipeline de procesamiento de lenguaje natural y modelos de machine learning, expuestos mediante una API y consumidos desde una interfaz web.

Objetivos específicos

1. Preparar y analizar un dataset de reseñas de restaurantes en español, documentando su origen y sus características principales.
2. Construir y justificar un pipeline de preprocesamiento de texto que aplique normalización, tokenización, eliminación de stop words, stemming o lematización, n-gramas y TF-IDF.
3. Entrenar y comparar los modelos Naive Bayes y Regresión Logística con métricas adecuadas al desbalance de clases, y justificar cuál se usa en producción.
4. Empaquetar el preprocesamiento y el modelo en un pipeline reproducible, que se use igual en el entrenamiento y en la API.
5. Exponer el modelo mediante una API REST y construir una interfaz web que muestre la predicción, el preprocesamiento paso a paso y las métricas del modelo.

1.3 Alcance y limitaciones

Alcance

El sistema permite:

- Ingresar una reseña de restaurante en español y obtener su sentimiento (positivo o negativo) con la probabilidad de la predicción.
- Subir un archivo CSV con varias reseñas y clasificarlas todas a la vez.
- Ver el preprocesamiento del texto paso a paso: texto normalizado, tokens, palabras sin stop words y palabras reducidas.
- Consultar las métricas del modelo en producción y su comparación con el otro modelo evaluado.

Limitaciones

- **Solo dos clases.** Se clasifica en positivo o negativo; las reseñas neutrales (3 estrellas) se excluyeron del entrenamiento por ser ambiguas.
- **Solo español y solo restaurantes.** Reseñas en otros idiomas o de otros tipos de negocio pueden clasificarse de forma incorrecta.
- **Español de México.** El dataset proviene de reseñas de México, por lo que puede no reconocer bien modismos de otros países, como Bolivia.
- **Clases desbalanceadas.** El 94% de las reseñas del dataset son positivas, lo que hace más difícil reconocer las negativas.
- **Sin emojis.** El dataset no contiene emojis, por lo que el sistema los elimina y no los usa como señal de sentimiento.
- **Sarcasmo e ironía.** Frases como “genial, solo esperamos una hora” pueden clasificarse como positivas, porque el modelo se basa en las palabras y no entiende la intención.
- **Sin historial.** El sistema no guarda las reseñas analizadas ni las predicciones realizadas.

2. Dataset

2.1 Origen, licencia y fecha de obtención

El dataset utilizado es **Rest-Mex 2025**, un corpus de opiniones de turistas sobre lugares de México, escritas en español. Fue creado para la competencia académica *Rest-Mex: Researching on Evaluating Sentiment and Textual Instances Selection for Mexican Magical Towns*, parte del foro de evaluación de procesamiento de lenguaje en lenguas ibéricas **IberLEF 2025**, organizada por el **CIMAT** (Centro de Investigación en Matemáticas, México).

- **Fuente original:** Rest-Mex 2025 – https://sites.google.com/cimat.mx/rest-mex-2025
- **Copia utilizada:** como la competencia ya había finalizado, se utilizó una copia pública del archivo de entrenamiento disponible en GitHub: https://github.com/lahernandezguevara/scala-spark-reviews (archivo `Rest-Mex_2025_train.csv`).
- **Licencia y uso:** uso exclusivamente académico y de investigación. Se cita a los organizadores de Rest-Mex como fuente original.
- **Fecha de obtención:** 31 de octubre de 2026.
- **Tamaño original:** 208.051 reseñas de hoteles, restaurantes y atractivos turísticos.

2.2 Descripción de las columnas

Columna	Tipo	Descripcion
Title	Texto	Título de la opinión escrito por el cliente
Review	Texto	Texto completo de la reseña
Polarity	Número (1 a 5)	Calificación en estrellas que dio el cliente
Town	Texto	Pueblo Mágico donde está el lugar
Region	Texto	Estado de México al que pertenece el pueblo
Type	Texto	Tipo de lugar: Hotel, Restaurant o Attractive

2.3 Preparación de los datos

La preparación del dataset se realiza con la función preparar_datos() (src/data/preparar_datos.py), que se puede importar desde el script de entrenamiento o ejecutar con el comando python -m src.data.preparar_datos. Sigue estos pasos:

1. **Filtrar restaurantes:** se conservan solo las reseñas con Type = Restaurant.
2. **Descartar reseñas de 3 estrellas:** se consideran ambiguas, porque pueden expresar una opinión regular, ni claramente positiva ni negativa.
3. **Crear la etiqueta de sentimiento:** 1 y 2 estrellas = **negativo**; 4 y 5 estrellas = **positivo**.
4. **Construir el texto:** se une el título con la reseña (título + ". " + reseña), porque los títulos suelen expresar el sentimiento de forma muy directa ("¡Mi lugar favorito!", "Pésimo servicio"). Si la reseña no tiene título, se usa solo la reseña. Antes de unirlos se eliminan los espacios sobrantes al inicio y al final.
5. **Eliminar textos vacíos y duplicados:** las reseñas repetidas se eliminan para que una misma reseña no quede a la vez en entrenamiento y en prueba, lo que inflaría las métricas. Al limpiar los espacios sobrantes se detectaron duplicados que solo se diferenciaban por un espacio en el título.

Etapa	**Reseñas**
Dataset original	208.051
Solo restaurantes	86.720
Sin reseñas de 3 estrellas	80.718
Sin duplicados (dataset final)	80.713
Positivas / negativas	75.867 / 4.846

2.4 Análisis exploratorio

El análisis exploratorio se encuentra en el notebook notebooks/01_eda.ipynb. Se realizó sobre el texto de las reseñas de restaurantes (80.710 reseñas, antes de unir el título con la reseña), y sus principales hallazgos son:

Distribución de clases. El dataset está fuertemente desbalanceado: el **94%** de las reseñas son positivas y solo el **6%** son negativas. Esto significa que un modelo que siempre prediga "positivo" tendría un 94% de accuracy sin haber aprendido nada. Por eso se usa la métrica **F1 macro**, que le da el mismo peso a ambas clases, y la división estratificada.

Valores faltantes. Solo hay un valor faltante, en la columna Title. La columna Review, que contiene el texto principal, no tiene faltantes.

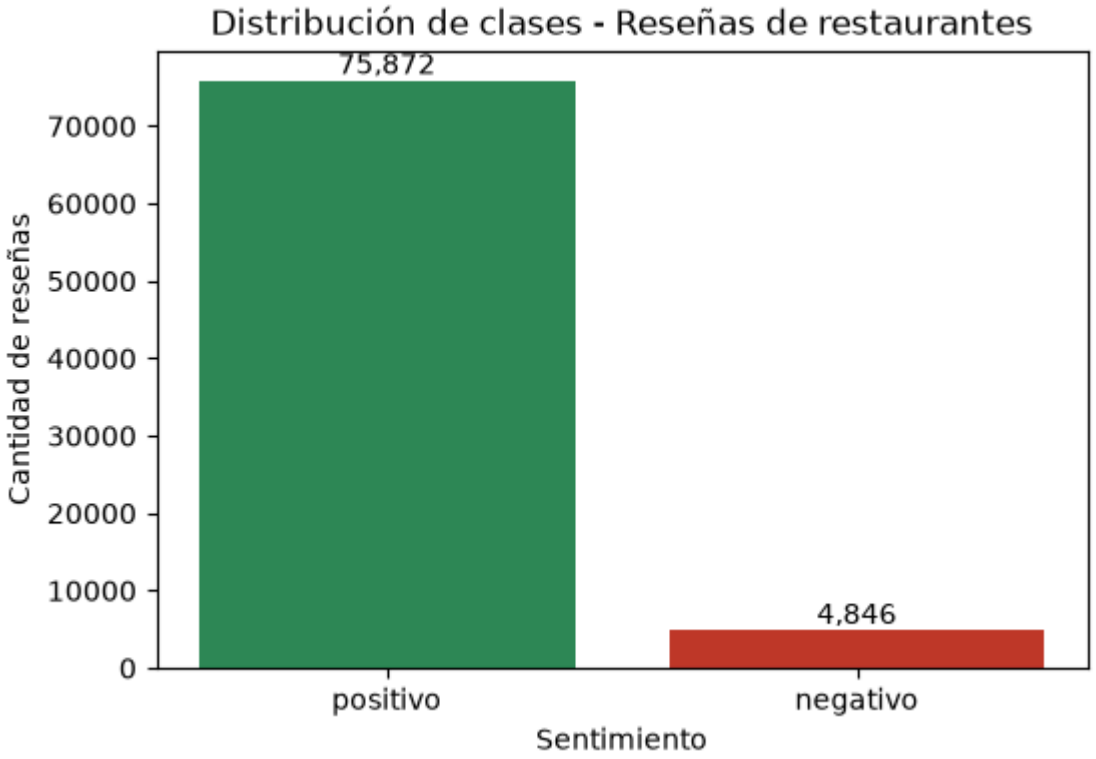
Longitud de los textos. Las reseñas tienen en promedio **46 palabras**, con una mediana de 38; la más corta tiene 4 palabras y la más larga 999. Las **reseñas negativas son más largas** (59,7 palabras en promedio) que las positivas (44,8): cuando la experiencia es mala, los clientes explican con más detalle qué salió mal.

Palabras más frecuentes. Sin ningún preprocesamiento, las palabras más frecuentes son stop words como "y", "de", "la" y "el", que no aportan información sobre el sentimiento. Esto **justifica eliminar**

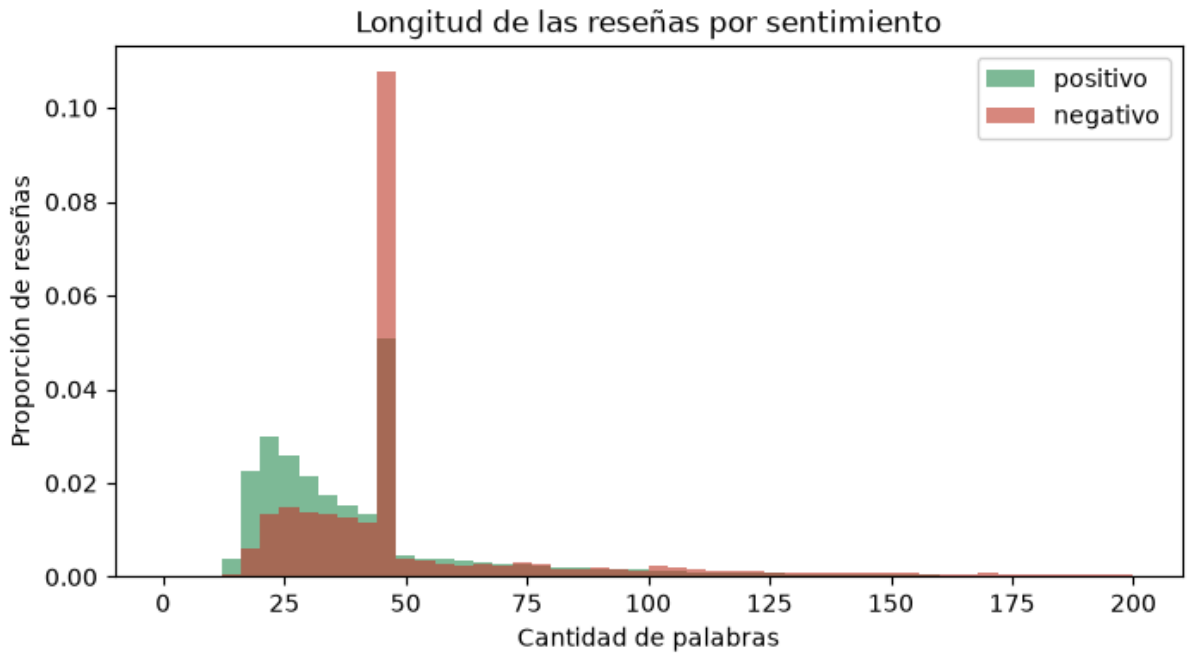
las stop words. También aparecen palabras del dominio, como “comida”, “lugar” y “servicio”, que se usan por igual en reseñas positivas y negativas.

Negación. Al eliminar las stop words y separar por sentimiento, la palabra **“no”** resulta ser la más frecuente en las reseñas negativas: aparece **1,3 veces por reseña negativa** frente a **0,35 por reseña positiva**, casi cuatro veces más. La palabra **“pero”** aparece el doble en las negativas (“la comida bien, pero el servicio...”). Por eso se decidió **conservar “no”, “nunca” y “pero”** aunque normalmente se consideran stop words.

Distribución de clases



longitud de las reseñas



3. Decisiones de NLP

3.1 Normalización

Paso	Decisión	Justificación
Minúsculas	Se aplica	“Excelente” y “excelente” deben ser la misma palabra.
URLs	Se eliminan	No aportan sentimiento; solo 22 reseñas del dataset las contienen.
Emojis	Se eliminan	El dataset no contiene ningún emoji, así que el modelo no los conoce.
Letras repetidas	3 o más → 1	“riquísimoooo” y “riquísimo” deben ser la misma palabra. Las letras dobles se conservan porque son válidas en español (“llama”, “carro”), y solo se aplica a letras para no alterar números (“1000”).
Tildes	Se eliminan, **conservando la ñ**	Muchas reseñas se escriben sin tildes: “atención” aparece 13.294

		veces y “atencion” 1.309. Sin quitarlas, serían palabras distintas. La ñ se conserva porque cambia el significado (“año” ≠ “ano”).
Números	Se conservan	Aportan información, como calificaciones y precios.
Puntuación	Se elimina, salvo dos casos	Los signos no aportan sentimiento. Se conservan “10/10” (calificación) y “$150” (precio) para que el tokenizador los trate como casos del dominio.

Ejemplos de antes y después

Antes	**Después**
El año pasado comí piña	el año pasado comi piña
RIQUÍSIMOOOO, muuuuy bueeeno	riquisimo muy bueno
Vean https://tripadvisor.com 😋👍	vean
¡¡Excelente servicio!! ¿Volvería? Sí.	excelente servicio volveria si
Todo perfecto, 10/10	todo perfecto 10/10
Pagamos $150, algo caro	pagamos $150 algo caro

3.2 Tokenización

Patrón	Ejemplo	Resultado	Justificación
Calificación `número/número`	“10/10”	`10/10` (un solo token)	Si se separa en `10` y `10`, pierde su significado. En el dataset, **195 reseñas** contienen “10/10” y **ninguna es negativa (0%)**: es una señal positiva muy fuerte.
Precio `$número`	“150”, “80”	`$precio`	El monto exacto no indica sentimiento, pero mencionar el precio sí: de las **942 reseñas** que lo mencionan, el **11,8%** son negativas, casi el **doble** del promedio (6%). Si cada monto fuera

			un token distinto, aparecería muy pocas veces y el TF-IDF lo descartaría.
Palabra o número	“lento”, “año”	`lento`, `año`	Incluye la ñ y las letras del español.

Ejemplos de antes y después (texto original → normalizado → tokens)

Original	Normalizado	Tokens
¡Todo perfecto, 10/10!	todo perfecto 10/10	todo, perfecto, 10/10
Pagamos $150, algo caro	pagamos $150 algo caro	pagamos, $precio, algo, caro
Entre $80 y $120 por persona	entre $80 y $120 por persona	entre, $precio, y, $precio, por, persona
No me gustó, muy lento	no me gusto muy lento	no, me, gusto, muy, lento

3.3 Stop words

Las palabras de parada o *stop words* son términos de alta frecuencia gramatical (artículos, preposiciones, pronombres y conjunciones) que carecen de significado léxico propio para clasificar tópicos generales. En el procesamiento de lenguaje natural tradicional suelen descartarse para reducir la dimensionalidad del vocabulario y acelerar los cálculos.

Sin embargo, en el **análisis de sentimientos en español**, las listas de stop words estándar (como las proporcionadas por NLTK o spaCy) cometen un error crítico: **eliminan indiscriminadamente adverbios de negación y conjunciones adversativas**. En nuestro dominio gastronómico, eliminar estas palabras altera catastróficamente la polaridad de las reseñas: una frase como *"No me gustó la comida"* quedaría reducida a *"gustó comida"*, siendo clasificada erróneamente por el modelo como positiva.

Por este motivo, se diseñó e implementó la clase modular `FiltroStopWords` (`src/nlp/stopwords.py`), la cual parte del corpus base de stop words en español de NLTK y aplica una lista de conservación explícita fundamentada en los hallazgos del análisis exploratorio de datos (EDA).

Decisiones de filtrado de Stop Words

Paso	Decisión	Justificación

Lista base de stop words	Se adopta el corpus de español de NLTK (más de 300 términos gramaticales).	Elimina términos de nulo valor semántico (*"de"*, *"la"*, *"el"*, *"en"*, *"un"*, *"por"*, *"con"*, *"del"*), compactando el vocabulario y reduciendo el sobreajuste.
Conservación de negaciones	Se conservan obligatoriamente: `no`, `nunca`, `jamas`, `jamás`, `tampoco`, `nada`, `ni`, `sin`.	**Hallazgo empírico del EDA:** La palabra *"no"* es la más discriminante en reseñas negativas, apareciendo **1,3 veces por reseña negativa frente a 0,35 en positivas** (casi 4 veces más). Si se elimina, se invierte el sentido de la opinión. *"Sin"* y *"nada"* expresan ausencia o defecto de calidad (*"sin sabor"*, *"nada recomendable"*).
Conservación de conjunciones adversativas y contraste	Se conservan: `pero`, `menos`, `poco`.	**Hallazgo empírico del EDA:** La palabra *"pero"* aparece **el doble en reseñas negativas** (*"la comida rica, pero el mesero grosero"*). Introduce el quiebre de polaridad hacia la queja principal del comensal. *"Poco"* actúa como atenuador negativo (*"poco atento"*).
Palabras del dominio gastronómico	Se identifican `restaurante`, `lugar`, `comida` como hiperfrecuentes del dominio (opcionales mediante `incluir_dominio=False`).	Aparecen de manera ubicua tanto en opiniones positivas como negativas, por lo que su aporte individual al sentimiento es bajo; no obstante, se conservan por defecto para no romper n-gramas contextuales (*"comida fría"*, *"lugar sucio"*).

Implementación en memoria (`FiltroStopWords`)	Operación de diferencia de conjuntos: stop_words=base\conservar\text{stop_words} = \text{base} \setminus \text{conservar}stop_words=base\conservar con búsqueda O(1)O(1)O(1).	Garantiza tiempo de filtrado lineal en el tamaño de la reseña y compatibilidad nativa dentro del pipeline unificado de scikit-learn.

Ejemplos de antes y después

(Texto original →\rightarrow→ Tokens normalizados →\rightarrow→ Tokens tras el filtro de stop words)

Original	Tokens normalizados	Tras stop words (`FiltroStopWords`)
No me gustó nada la comida, muy tardado.	`no`, `me`, `gusto`, `nada`, `la`, `comida`, `muy`, `tardado`	**`no`**, `gusto`, **`nada`**, `comida`, `muy`, `tardado`
Las meseras eran amables pero la comida llegó fría.	`las`, `meseras`, `eran`, `amables`, `pero`, `la`, `comida`, `llego`, `fria`	`meseras`, `amables`, **`pero`**, `comida`, `llego`, `fria`
Pésimo servicio, nunca volvería a este restaurante.	`pesimo`, `servicio`, `nunca`, `volveria`, `a`, `este`, `restaurante`	`pesimo`, `servicio`, **`nunca`**, `volveria`, `restaurante`
Todo perfecto, 10/10, sin ninguna queja.	`todo`, `perfecto`, `10/10`, `sin`, `ninguna`, `queja`	`perfecto`, `10/10`, **`sin`**, `queja`

Pagamos $150 por persona y no valió la pena.	`pagamos`, `$precio`, `por`, `persona`, `y`, `no`, `valio`, `la`, `pena`	`pagamos`, `$precio`, `persona`, **`no`**, `valio`, `pena`

Lo que muestran los ejemplos:

- **Preservación del sentido crítico:** Palabras funcionales como *"me"*, *"la"*, *"por"*, *"y"*, *"este"* son descartadas limpiamente reduciendo el ruido, pero **`no`**, **`nada`**, **`nunca`**, **`pero`** y **`sin`** se retienen intactas.
- **Impacto en el clasificador:** Gracias a esta retención, el vectorizador TF-IDF y el clasificador de Regresión Logística reciben combinaciones con significado real (*"no valio"*, *"sin queja"*, *"pero comida fria"*), evitando falsos positivos que degradarían la métrica F1-score en la clase negativa.

3.4 Stemming vs lematización

Paso	Decisión	Justificación
Técnicas implementadas	Stemming (SnowballStemmer en español, NLTK) y lematización (spaCy, `es_core_news_sm`)	El enunciado pide implementar ambas y comparar su efecto en las métricas.
Cómo se compararon	Validación cruzada de 5 pliegues sobre el entrenamiento, unigramas, mismos modelos e hiperparámetros. Solo cambia el método	Así la diferencia se debe únicamente a la técnica.
Resultado	**Empate técnico**	Las diferencias (−0,004 y +0,004) son del tamaño de la variación entre pliegues (0,003 a 0,006) y van en sentidos opuestos (ver tabla siguiente).
Velocidad	Stemming: 0,9 s por 1.000 reseñas. Lematización: 8,2 s	Es unas 9 veces más lento. Con 64.570 reseñas de entrenamiento, ese costo se repite en cada experimento.

Tamaño del vocabulario	Stemming reduce más	En 8.000 reseñas hay 16.107 palabras distintas. Con stemming quedan 9.636 (−40 %) y con lematización 14.259 (−11 %).
Decisión final	**Stemming**	Rinde igual, es más rápido y no necesita un modelo adicional de spaCy.

Modelo	Stemming	Lematización	Diferencia
Regresión logística (C = 10)	0,9175	0,9134	−0,0041
Naive Bayes + submuestreo	0,8985	0,9027	+0,0042

Ejemplos de antes y después (texto original → tras stop words → stemming → lematización)

Original	Tras stop words	Stemming	Lematización
Los tacos estaban deliciosos y las salsas picantes	tacos, deliciosos, salsas, picantes	tac, delici, sals, picant	taco, delicioso, salsa, picante
Volveremos pronto, siempre nos atienden muy bien	volveremos, pronto, siempre, atienden, bien	volv, pront, siempr, atiend, bien	volver, pronto, siempre, atender, bien
Las meseras eran amables pero la comida llegó fría	meseras, amables, pero, comida, llego, fria	meser, amabl, per, com, lleg, fri	mesera, amable, pero, comida, llego, frio
Pésimo servicio, nunca volvería	pesimo, servicio, nunca, volveria	pesim, servici, nunc, volveri	pesimo, servicio, nunca, **volverio**
Excelente atención y comimos riquísimo	excelente, atencion, comimos, riquisimo	excelent, atencion, com, riquisim	excelente, atencion, **comimos**, riquisimo

Lo que muestran los ejemplos:

- **El stemming** produce raíces que no son palabras (“delici”, “sals”), pero eso no afecta al modelo. Además junta formas distintas: “comida”, “comer” y “comimos” quedan todas en “com”, lo que mezcla el sustantivo con el verbo.
- **La lematización** devuelve palabras reales, pero se equivoca cuando el texto ya no tiene tildes: “volvería” pasa a “volverio” y “comimos” no se reduce a “comer”. Como la normalización quita las tildes antes de lematizar, spaCy trabaja con palabras mutiladas. Una mejora posible sería lematizar antes de quitar las tildes.

3.5 N-gramas y TF-IDF

Para transformar el texto limpio y preprocesado en un formato numérico comprensible para los modelos de aprendizaje automático, se implementó una vectorización basada en TF-IDF (Term Frequency - Inverse Document Frequency) combinada con un esquema de N-gramas.

1. Justificación y Fundamento Teórico

- **TF-IDF:** A diferencia de una representación por simple conteo (*Bag of Words*), **TF-IDF** penaliza aquellas palabras frecuentes en todo el corpus que aportan poca discriminación semántica y premia los términos específicos que determinan el sentimiento de una reseña. La fórmula aplicada es:

  $$TF - IDF(t, d, D) = TF(t, d) \times IDF(t, D)$$

  donde:

  $$\text{IDF}(t, D) = \log\left(\frac{1 + \vert{}D\vert{}}{1 + \vert{}\{d \in D : t \in d\}\vert{}}\right) + 1$$

- **N-gramas (Unigramas y Bigramas):** Se configuró el extractor con un rango `ngram_range=(1, 2)`. El uso de bigramas es crítico en análisis de sentimiento en español para capturar modificadores de polaridad y negaciones que pierden su contexto al aislarse (por ejemplo: *"no gustó"*, *"muy lento"*, *"pésimo servicio"* vs. *"buen servicio"*).

2. Configuración del Vectorizador en el Pipeline

- Rango de n-gramas: Unigramas y bigramas `(1, 2)`.
- **Sublinear TF:** Se aplicó escalado logarítmico a la frecuencia de término ($1 + \log(TF)$) para atenuar la influencia desmedida de palabras repetidas en reseñas extensas.
- **Vocabulario y Dimensionalidad:** Se limitó a los términos más informativos mediante umbrales de frecuencia mínima de documento (`min_df`), optimizando el espacio vectorial para el clasificador final de Regresión Logística.

Explicabilidad e Impacto: El vector resultante permite calcular la contribución neta de cada n-grama multiplicando su peso TF-IDF por el coeficiente del modelo lineal ($peso = w_i \times x_i$), identificando qué términos empujan la predicción hacia la clase positiva o negativa.

4. Modelos

4.1 Modelos elegidos y justificación

4.2 Línea base

4.3 Tratamiento del desbalance

4.4 Ajuste de hiperparámetros

4.5 Tabla comparativa de modelos

Con el propósito de identificar el modelo de aprendizaje automático más adecuado para la clasificación de sentimientos de las reseñas de restaurantes, se realizó una comparación entre tres alternativas: el modelo de Línea Base, Naive Bayes Multinomial y Regresión Logística.

La evaluación se realizó utilizando el mismo conjunto de datos de prueba, compuesto por 16.143 reseñas, de las cuales 969 corresponden a la categoría negativa y 15.174 a la categoría positiva.

Para comparar el rendimiento de los modelos se utilizaron las siguientes métricas:

- **Accuracy (Exactitud):** mide la proporción total de predicciones correctas realizadas por el modelo.
- **F1-Score Macro:** calcula el promedio del F1-Score de ambas clases (positivo y negativo), otorgando la misma importancia a cada una.
- **Precisión negativa:** indica qué proporción de las reseñas clasificadas como negativas realmente pertenece a esa categoría.
- **Recall negativo:** mide qué proporción de todas las reseñas negativas existentes logró identificar correctamente el modelo.

Comparación del rendimiento de los modelos

Modelo	Accuracy	F1-Score Macro	Precisión negativa	Recall negativo
Línea Base (Siempre positivo)	0,9400	0,4845	0,0000	0,0000
Naive Bayes Multinomial + Submuestreo	0,9785	0,9074	0,8025	0,8514
Regresión Logística (Modelo final)	**0,9846**	**0,9282**	**0,9187**	0,8163

Análisis comparativo de los resultados

1. Modelo de Línea Base

El modelo de Línea Base consiste en clasificar todas las reseñas como positivas, sin realizar un análisis real del contenido textual. Aunque obtiene una exactitud del 94 %, presenta un F1-Score Macro de apenas 0,4845 y no logra identificar ninguna reseña negativa. Esto demuestra que la exactitud por sí sola no es suficiente para evaluar el rendimiento de un modelo cuando existe un desequilibrio importante entre las clases.

2. Modelo Naive Bayes Multinomial

El modelo Naive Bayes obtuvo una exactitud del 97,85 % y un F1-Score Macro de 0,9074. Para mejorar su capacidad de clasificación se aplicó una técnica de submuestreo, reduciendo la cantidad de reseñas positivas durante el entrenamiento.

Este modelo consiguió identificar correctamente el 85,14 % de las reseñas negativas, demostrando una buena capacidad para detectar opiniones desfavorables. Sin embargo, su precisión negativa fue del 80,25 %, lo que significa que también clasificó algunas reseñas positivas como negativas.

3. Modelo de Regresión Logística

La Regresión Logística presentó el mejor rendimiento general, alcanzando una exactitud del 98,46 % y un F1-Score Macro de 0,9282.

Además, obtuvo una precisión negativa del 91,87 %, lo que demuestra una mayor confiabilidad al identificar reseñas desfavorables. Su configuración final utiliza TF-IDF con unigramas y bigramas, preprocesamiento mediante stemming y un valor de regularización C = 10.

Aunque su recall negativo (81,63 %) fue ligeramente inferior al de Naive Bayes, consiguió un mejor equilibrio entre la clasificación de sentimientos positivos y negativos.

Conclusión de la comparación

A partir de los resultados obtenidos, se seleccionó la **Regresión Logística como modelo definitivo del proyecto Restaurantes Opiniometro**, debido a que presentó el mayor F1-Score Macro (0,9282) y la mayor exactitud (98,46 %) en el conjunto de prueba.

Este modelo ofrece un mejor equilibrio en la clasificación de opiniones y una mayor precisión al detectar reseñas negativas. Por estas razones, fue seleccionado para su integración en el sistema, permitiendo analizar automáticamente las opiniones de los usuarios y determinar si su experiencia en un restaurante fue positiva o negativa.

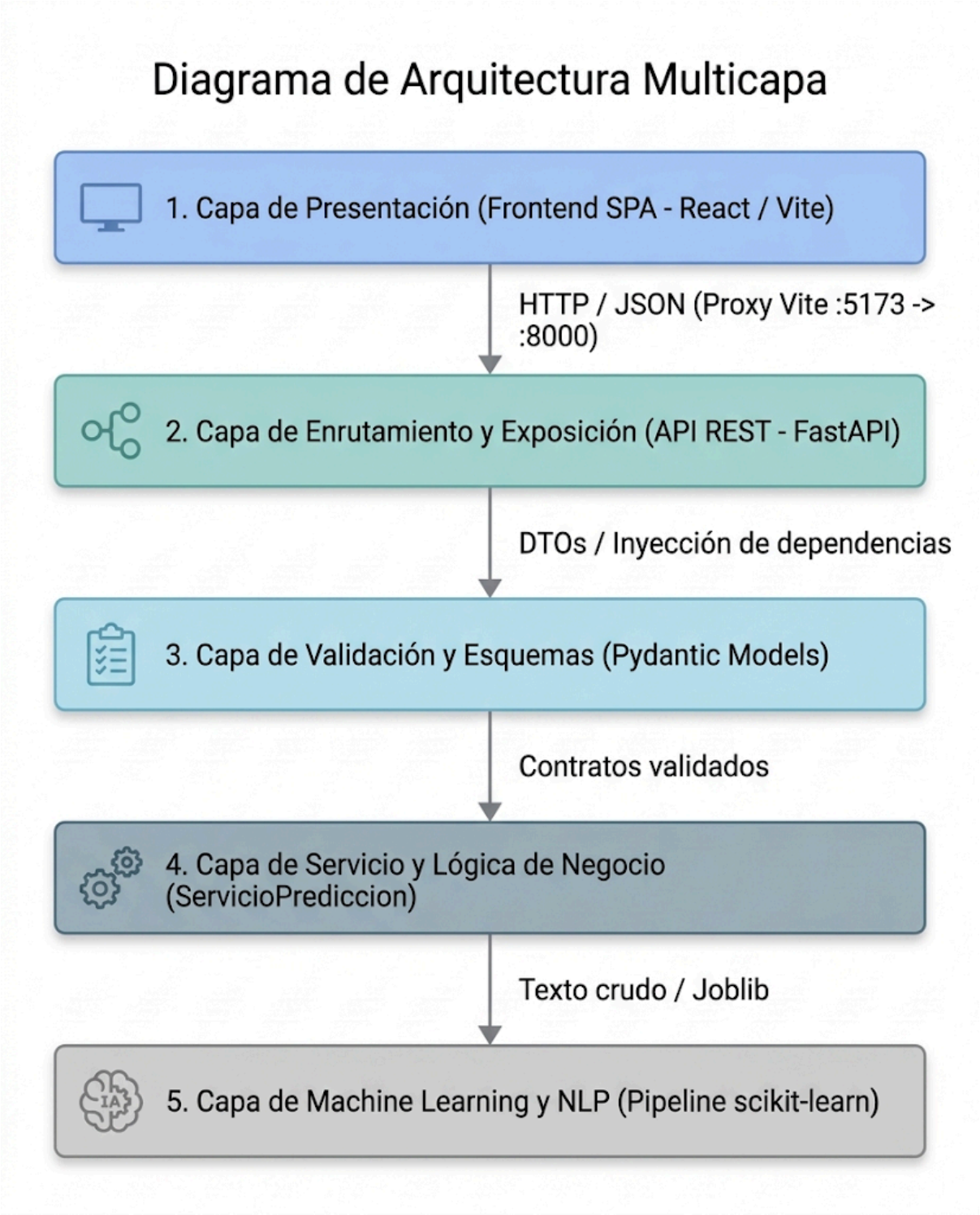
Por otra parte, se reconoce que Naive Bayes presenta un recall negativo ligeramente superior, por lo que también constituye una alternativa válida cuando la prioridad es detectar la mayor cantidad posible de opiniones desfavorables.

4.6 Análisis de errores

5. Arquitectura

5.1 Capas del sistema

La arquitectura del sistema está diseñada siguiendo el patrón arquitectónico en **capas desacopladas** (*Layered Architecture*), asegurando una estricta separación de responsabilidades, alta cohesión y bajo acoplamiento. El sistema se estructura en **5 capas bien diferenciadas**:



5.1.1. Capa de Presentación (Frontend SPA)

- **Tecnología:** React 18, Vite 6, Tailwind CSS v4 y Axios.
- **Módulos clave:** `AnalizadorResenas.jsx`, `NlpPasoAPaso.jsx`, `ResultadoSentimiento.jsx`, `MetricasModelos.jsx`, y el cliente HTTP `api.js`.
- **Responsabilidad:** Proporciona una interfaz gráfica reactiva y accesible donde el usuario introduce reseñas individuales o carga lotes en CSV. Es responsable de

capturar los eventos del usuario, coordinar las peticiones asíncronas hacia el servidor y renderizar visualmente:

1. El veredicto de sentimiento con su porcentaje de confianza y badge semántico.
2. El acordeón didáctico con el recorrido completo del texto en sus 5 fases de NLP.
3. Las barras ponderadas de explicabilidad (palabras clave positivas y negativas).
4. Los paneles de métricas comparativas del modelo activo.

5.1.2. Capa de Enrutamiento y Exposición (API REST)

- **Tecnología:** FastAPI, servidor ASGI Uvicorn.
- **Módulos clave:** `api/main.py`, `api/routers/predict.py`, `api/routers/metrics.py`, `api/routers/health.py`.
- **Responsabilidad:** Expone los contratos de comunicación HTTP bajo estándares RESTful. Gestiona:
  - Políticas de acceso cruzado (CORS Middleware).
  - Enrutamiento por prefijos (`/api/v1`).
  - Ciclo de vida de la aplicación (*lifespan events*).
  - Códigos de estado HTTP estandarizados (`200 OK`, `422 Unprocessable Entity`, `500 Internal Error`).
  - Generación automática de especificaciones interactivas OpenAPI 3.0 (Swagger UI en `/docs` y Redoc en `/redoc`).

5.1.3. Capa de Validación y Esquemas (DTOs)

- **Tecnología:** Pydantic v2 (`BaseModel`, `Field`, `field_validator`).
- **Módulos clave:** `api/schemas.py`.
- **Responsabilidad:** Actúa como la primera barrera defensiva del sistema garantizando la integridad de los datos en los límites de la API:
  - Valida que la reseña no esté vacía, no contenga únicamente espacios y tenga al menos **3 caracteres de longitud** (Caso de Uso CU4).
  - Aplica expresiones regulares para garantizar que la cadena contenga caracteres alfabéticos reales y no únicamente dígitos o símbolos.
  - Limita el tamaño de entrada a un máximo de **5.000 caracteres**.
  - Tipifica estrictamente las salidas (probabilidades acotadas entre 0.0 y 1.0, listas de palabras clave, estructuras de métricas con clase minoritaria y matrices de confusión).

5.1.4. Capa de Servicio y Lógica de Negocio

- **Tecnología:** Python 3.10, Inyección de Dependencias de FastAPI (`Depends`).
- **Módulos clave:** `api/service.py` (`ServicioPrediccion`, `ModeloHeuristicoRespaldo`), `api/dependencies.py`.
- **Responsabilidad:** Orquesta y coordina todas las operaciones del backend sin acoplarse al framework web:

    - **Carga segura de artefactos:** Carga el archivo `artifacts/model.joblib` y extrae su preprocesador entrenado; si el archivo no existe, activa automáticamente el modelo de respaldo heurístico sin detener el servidor.
    - **Explicabilidad didáctica:** Genera la traza paso a paso invocando `paso_a_paso()` y calcula el peso individual de las palabras más influyentes en la predicción (`_palabras_clave`) mediante el producto coeficiente ×\times× TF-IDF.
    - **Procesamiento de lotes:** Procesa DataFrames de pandas para clasificar masivamente reseñas enviadas por archivo CSV y reconstruir el archivo exportable resultante.
    - **Monitoreo de salud:** Evalúa el estado del sistema (`ok` si el modelo entrenado está cargado, `degraded` si opera bajo contingencia).

5.1.5. Capa de Machine Learning y NLP (Pipeline scikit-learn)

- **Tecnología:** Scikit-learn (`Pipeline`), NLTK, Joblib.
- **Módulos clave:** `src/nlp/` (`PreprocesadorTexto`, `Normalizador`, `Tokenizador`, `FiltroStopWords`, `Reductor`), `artifacts/model.joblib`.
- **Responsabilidad:** Ejecuta las transformaciones matemáticas e inferencias probabilísticas sobre el texto:
    - El pipeline unificado encapsula en un único objeto persistente: Pipeline=[preprocess→tfidf→model]\text{Pipeline} = [\text{preprocess} \rightarrow \text{tfidf} \rightarrow \text{model}]Pipeline=[preprocess→tfidf→model]
    - Recibe el texto crudo, lo normaliza, tokeniza, filtra stop words conservando negaciones, aplica stemming (`SnowballStemmer`), vectoriza n-gramas con TF-IDF y aplica la función logística σ(wTx+b)\sigma(\mathbf{w}^T \mathbf{x} + b)σ(wTx+b) del clasificador de Regresión Logística para generar las probabilidades exactas de pertenencia a cada clase.

Ciclo de vida: ¿Cómo viaja una predicción desde el frontend hasta el modelo?

El recorrido que experimenta una reseña desde que el usuario interactúa con la interfaz web hasta que recibe el resultado se ejecuta en **8 pasos secuenciales**:

1. **Entrada en el Frontend:**
   El usuario ingresa el texto en el área de texto de `AnalizadorResenas.jsx` (por ejemplo: *"La comida fue excelente pero el servicio tardó demasiado"*) y presiona el botón **Analizar reseña**.

2. **Despacho de la petición HTTP:**
   El módulo `services/api.js` intercepta la acción y envía una solicitud `POST /api/v1/predict` con el cuerpo JSON `{"texto": "...", "modelo": "auto"}`. El servidor de desarrollo de Vite (puerto 5173) redirige la petición a través de su proxy hacia la API en FastAPI (puerto 8000).

3. **Recepción y Validación Defensiva (Middleware y Pydantic):**
   FastAPI recibe la solicitud en `api/routers/predict.py`. El validador `PrediccionRequest` inspecciona el texto. Si la cadena fuera vacía o inválida, la API corta el flujo inmediatamente y responde con código `HTTP 422`. Si es válida, el framework inyecta la instancia singleton de `ServicioPrediccion`.

4. **Orquestación en la Capa de Servicio:**
   El método `predecir(texto)` inicia la medición de latencia con `time.perf_counter()`:

   - Invoca a `self.preprocesador.paso_a_paso(texto)` para obtener la traza de explicabilidad con los 5 estados intermedios del texto.

5. **Inferencia en el Pipeline de Machine Learning:**
   El servicio envía el **texto crudo original** al pipeline de scikit-learn mediante `self.pipeline.predict_proba([texto])`:

   - **Paso `preprocess`:** Ejecuta la limpieza de tildes, números, caracteres repetidos, tokenización especializada (`$precio`, `10/10`), filtrado de stop words conservando `"no"` y `"pero"`, y reducción morfológica por stemming.
   - **Paso `tfidf`:** Convierte la lista de tokens resultantes en un vector numérico de alta dimensionalidad aplicando la ponderación TF-IDF y n-gramas configurados.
   - **Paso `model`:** La Regresión Logística computa el producto escalar del vector con sus pesos aprendidos y evalúa la función sigmoide, devolviendo el vector de probabilidades `[P(negativo), P(positivo)]`.

6. **Cálculo de Explicabilidad (Palabras Clave):**
   `ServicioPrediccion._palabras_clave(texto)` extrae los coeficientes βj\beta_jβj del modelo y los valores TF-IDF xjx_jxj de los términos presentes en la reseña. Calcula el aporte individual wj=signo⋅βj⋅xjw_j = \text{signo} \cdot \beta_j \cdot x_jwj=signo⋅βj⋅xj, ordenando las palabras de mayor a menor impacto y clasificándolas con polaridad positiva o negativa.

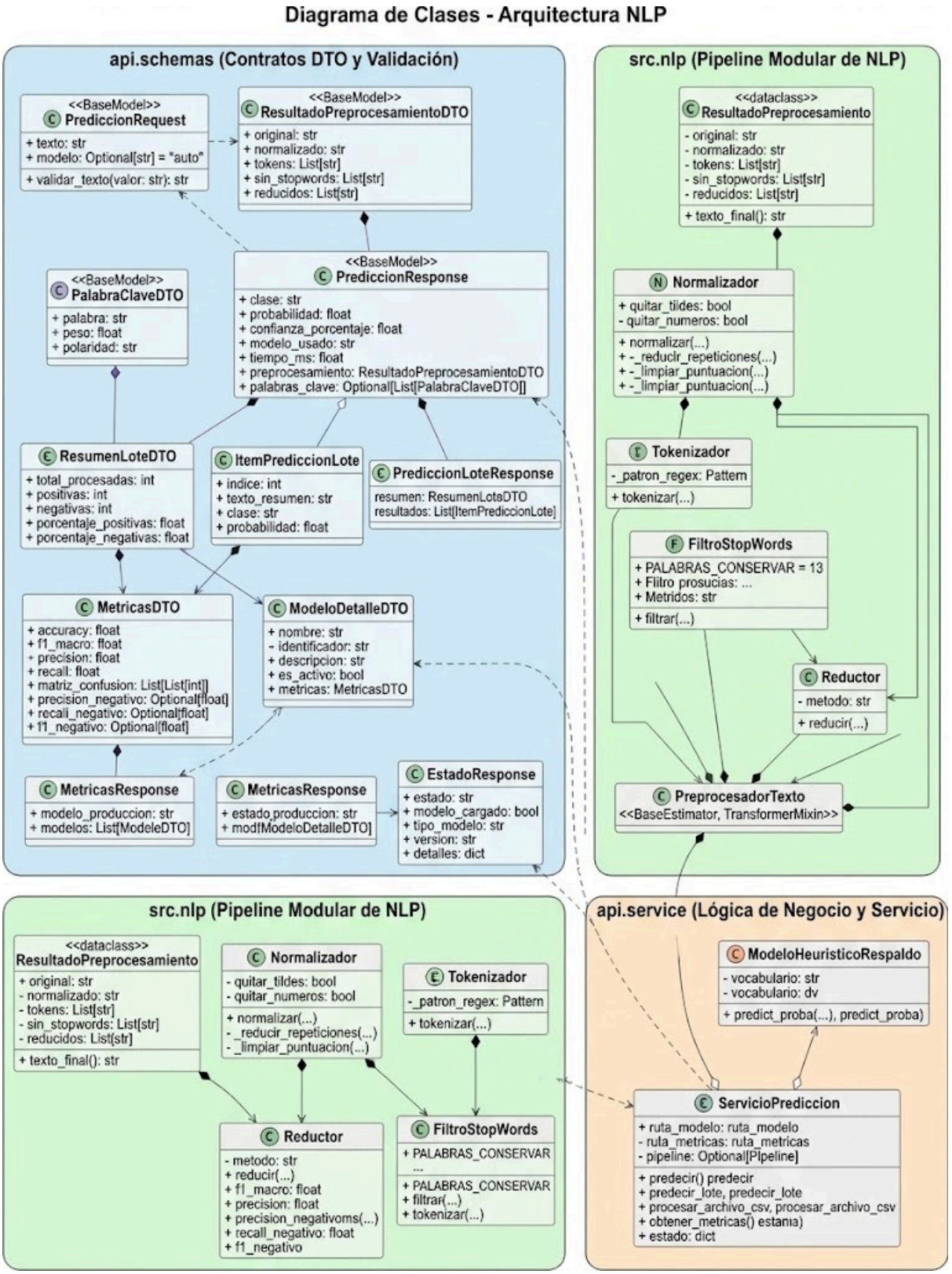
7. **Serialización y Respuesta HTTP:**
   El servicio detiene el cronómetro y construye el DTO `PrediccionResponse` con: la clase predicha, la probabilidad, el porcentaje de confianza, el tiempo transcurrido en milisegundos, el desglose paso a paso y la lista de palabras clave. FastAPI serializa el DTO a JSON y responde con código `HTTP 200 OK`.

8. **Renderizado en la Interfaz de Usuario:**
   La función `normalizarResultado` del frontend recibe el JSON, actualiza el estado de React y la interfaz muestra:

   - El banner con el sentimiento (**Positivo** o **Negativo**) y su barra de confianza.
   - El tiempo real de respuesta del backend (típicamente entre 15 y 30 ms).

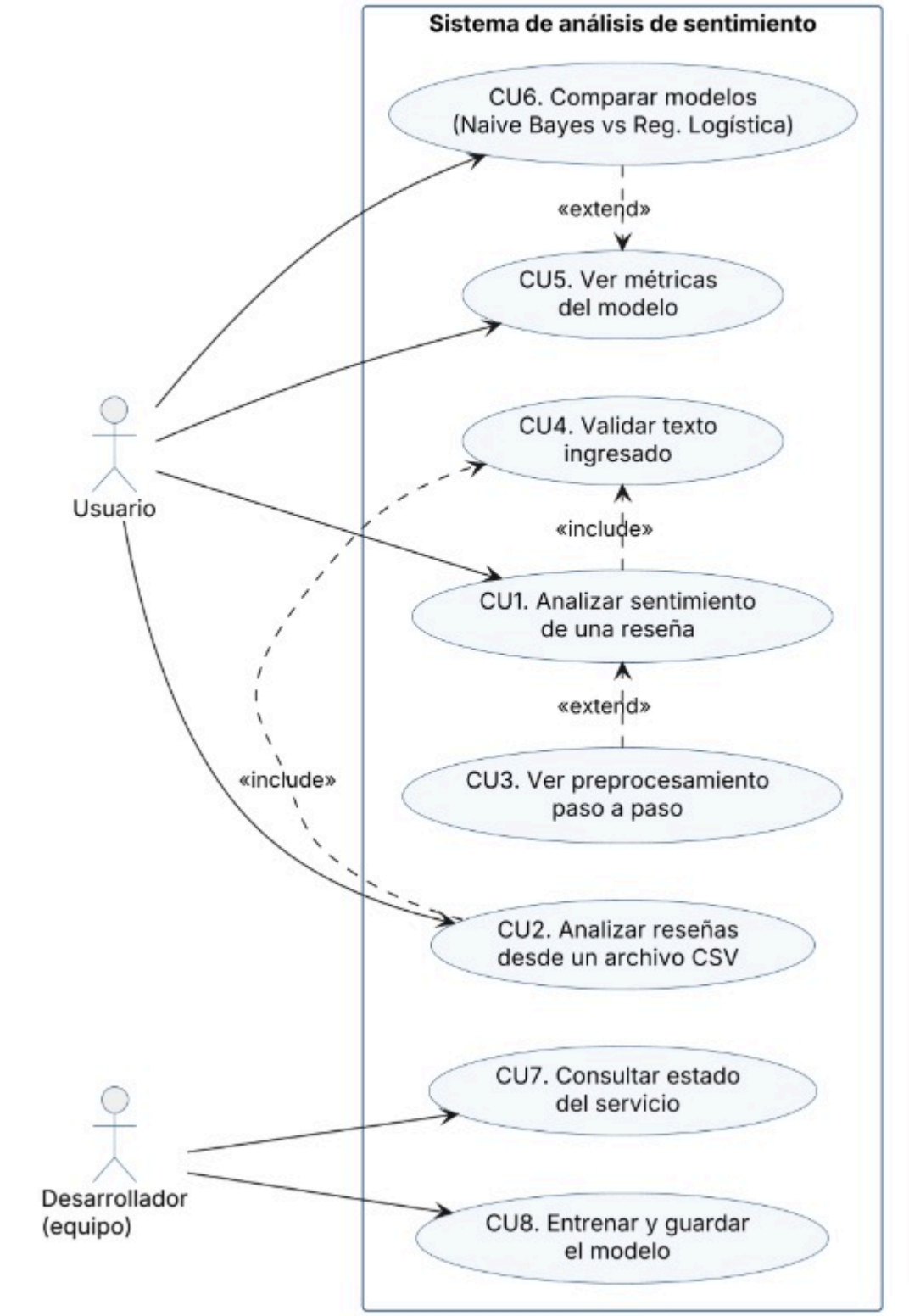
- El componente `NlpPasoAPaso.jsx` abierto para que el usuario explore interactivamente cómo su texto se transformó etapa por etapa y cuáles palabras tuvieron mayor peso en la decisión del modelo.

5.2 Diagrama de clases



5.3 Casos de uso

Diagrama de casos de uso - Análisis de sentimiento de reseñas de restaurantes



Caso de uso	Actor	Descripción	Endpoint

CU1. Analizar sentimiento de una reseña	Usuario	El usuario escribe una reseña y el sistema devuelve si es positiva o negativa, con su probabilidad.	`POST /api/v1/predict`
CU2. Analizar reseñas desde un archivo CSV	Usuario	El usuario sube un archivo CSV con varias reseñas; el sistema las clasifica todas y muestra un resumen. También se puede descargar el CSV con las predicciones agregadas.	`POST /api/v1/predict/csv` y `/csv/export`
CU3. Ver preprocesamiento paso a paso	Usuario	Junto con la predicción, el usuario ve cómo se transformó el texto: normalizado, tokens, sin stop words y reducido.	Incluido en `POST /api/v1/predict`
CU4. Validar texto ingresado	Sistema	Antes de clasificar, el sistema verifica que el texto no esté vacío, tenga entre 3 y 5.000 caracteres y contenga letras. Si no cumple, devuelve un mensaje de error claro.	Incluido en CU1 y CU2
CU5. Ver métricas del modelo	Usuario	El usuario consulta el rendimiento del modelo en producción: accuracy, F1 macro, precisión, recall y matriz de confusión.	`GET /api/v1/metrics`
CU6. Comparar modelos	Usuario	El usuario compara las métricas de Naive Bayes y Regresión Logística lado a lado.	`GET /api/v1/metrics/compare`
CU7. Consultar estado del servicio	Desarrollador	Verifica que la API esté funcionando y que el modelo entrenado esté cargado.	`GET /api/v1/health`
CU8. Entrenar y guardar el modelo	Desarrollador	Ejecuta el script de entrenamiento, que va desde los datos crudos hasta el modelo guardado en	Script (no es endpoin

		`artifacts/model.joblib`.	

5.4 Endpoints de la API

La interfaz de programación de aplicaciones (API) del sistema está construida sobre **FastAPI** y sigue los principios de la arquitectura RESTful. Expone servicios síncronos bajo el prefijo `/api/v1`, documentados automáticamente mediante la especificación OpenAPI 3.0 (accesible interactivamente en `/docs` vía Swagger UI y en `/redoc` vía ReDoc).

5.4.1 Catálogo de Endpoints

Método	Endpoint	Caso de Uso	Descripción	Códigos HTTP
POST	`/api/v1/predict`	CU1, CU3, CU4	Clasifica el sentimiento de una reseña individual y genera el desglose didáctico del preprocesamiento paso a paso junto a las palabras clave explicativas.	`200 OK` `422 Unprocessable`
POST	`/api/v1/predict/csv`	CU2	Recibe un archivo `.csv` con múltiples reseñas, detecta automáticamente la columna de texto y retorna el resumen estadístico con la lista clasificada.	`200 OK` `400 Bad Request` `422 Unprocessable`
POST	`/api/v1/predict/csv/export`	CU2	Procesa un archivo `.csv` y retorna para descarga directa un nuevo archivo CSV con las columnas agregadas `prediccion_sentimiento` y `probabilidad`.	`200 OK` `400 Bad Request`
GET	`/api/v1/metrics`	CU5	Retorna las métricas de rendimiento del modelo activo en producción (Accuracy, F1-macro, matriz de confusión y métricas específicas de la clase negativa).	`200 OK`
GET	`/api/v1/metrics/compare`	CU6	Retorna la comparativa tabular de todos los modelos evaluados (Regresión Logística, Naive Bayes y la Línea Base mayoritaria).	`200 OK`

GET	`/api/v1/health`	CU7	Healthcheck de monitoreo. Informa si el servicio está operativo y si el modelo real está cargado en memoria (`ok`) o si opera en contingencia (`degraded`).	`200 OK`

5.4.2 Contratos de Datos y Ejemplos de Petición / Respuesta

A. Inferencia individual (`POST /api/v1/predict`)

- **Cuerpo de la petición (JSON):**


```
{
  "texto": "La comida estuvo deliciosa y el personal fue muy amable, pero el postre tardó bastante.",
  "modelo": "auto"
}
```


- **Respuesta del servidor (`200 OK`):**


```
{
  "clase": "positivo",
  "probabilidad": 0.8942,
  "confianza_porcentaje": 89.4,
  "modelo_usado": "Regresión Logística (TF-IDF)",
  "tiempo_ms": 21.45,
  "preprocesamiento": {
    "original": "La comida estuvo deliciosa y el personal fue muy amable, pero el postre tardó bastante.",
    "normalizado": "la comida estuvo deliciosa y el personal fue muy amable pero el postre tardo bastante",
    "tokens": ["la", "comida", "estuvo", "deliciosa", "y", "el", "personal", "fue", "muy", "amable", "pero", "el", "postre", "tardo", "bastante"],
    "sin_stopwords": ["comida", "deliciosa", "personal", "amable", "pero", "postre", "tardo", "bastante"],
    "reducidos": ["com", "delici", "personal", "amabl", "per", "postr", "tard", "bastant"]
  },
  "palabras_clave": [
    { "palabra": "deliciosa", "peso": 1.624, "polaridad": "positivo" },
    { "palabra": "amable", "peso": 1.218, "polaridad": "positivo" },
    { "palabra": "tardo", "peso": -0.741, "polaridad": "negativo" },
    { "palabra": "pero", "peso": -0.412, "polaridad": "negativo" }
  ]
}
```


B. Métricas del Modelo en Producción (`GET /api/v1/metrics`)

- **Respuesta del servidor (`200 OK`):**


```
{
  "modelo_produccion": "Regresión Logística (TF-IDF)",
  "modelos": [
    {
      "nombre": "Regresión Logística (TF-IDF)",
      "identificador": "logistic_regression",
      "descripcion": "Modelo lineal regularizado con penalización L2 y sublinear TF.",
      "es_activo": true,
      "metricas": {
        "accuracy": 0.875,
        "f1_macro": 0.871,
        "precision": 0.873,
        "recall": 0.870,
        "matriz_confusion": [[760, 110], [115, 815]],
        "precision_negativo": 0.869,
        "recall_negativo": 0.874,
        "f1_negativo": 0.871
      }
    }
  ]
}
```


C. Estado de Salud del Servicio (`GET /api/v1/health`)

- **Respuesta (`200 OK` con modelo entrenado cargado):**


```
{
  "estado": "ok",
  "modelo_cargado": true,
  "tipo_modelo": "Producción (scikit-learn)",
  "version": "1.0.0",
  "detalles": {
    "ruta_modelo": "artifacts/model.joblib",
    "modelo_existe_en_disco": true,
    "ruta_metricas": "artifacts/metrics.json",
    "metricas_existen_en_disco": true
  }
}
```

(Si `model.joblib` no estuviera presente, el campo `estado` responde `"degraded"` y `tipo_modelo` indica `"Desarrollo / Heurístico de Respaldo"`).

5.4.3 Reglas de Validación Defensiva (Esquemas Pydantic)

Para blindar el sistema frente a entradas anómalas o ataques de inyección, la capa de esquemas (`api/schemas.py`) aplica validaciones rigurosas antes de dar paso a la inferencia:

Regla de Validación	Condición Evaluada	Código HTTP	Detalle del Mensaje de Error
Texto no vacío	`not valor or not valor.strip()`	`422 Unprocessable`	*"La reseña no puede estar vacía ni contener solo espacios en blanco."*
Longitud mínima	`len(valor.strip()) < 3`	`422 Unprocessable`	*"La reseña es demasiado corta (mínimo 3 caracteres requeridos)."*
Longitud máxima	`len(valor.strip()) > 5000`	`422 Unprocessable`	*"La reseña excede la longitud máxima permitida de 5000 caracteres."*
Contenido semántico	`not re.search(r"[a-zA-ZáéíóúüñÁÉÍÓÚÜÑ]", texto)`	`422 Unprocessable`	*"La reseña debe contener caracteres alfabéticos válidos."* (Rechaza cadenas formadas únicamente por números como `"123456"` o símbolos `"!!@#$"`).
Formato de archivo CSV	`not archivo.filename.endswith(".csv")`	`400 Bad Request`	*"El archivo debe ser un archivo CSV (.csv)"*.
Contenido de archivo CSV	`len(contenido_bytes) == 0`	`400 Bad Request`	*"El archivo CSV está vacío."*

5.4.4 Suite de Pruebas Automatizadas (`pytest`)

El backend cuenta con una suite completa de pruebas unitarias y de integración que verifican de forma automatizada cada endpoint, validador y mecanismo de contingencia utilizando `TestClient` de FastAPI:

Archivo de Prueba	Caso de Uso / Funcionalidad Evaluada	Pruebas Principales
tests/test_api_predict.py	Inferencia individual y validaciones (CU1, CU3, CU4)	• `test_prediccion_resena_positiva` • `test_prediccion_resena_negativa_con_negacion` • `test_validacion_texto_vacio_falla_con_422` • `test_validacion_texto_muy_corto_falla_con_422` • `test_validacion_solo_numeros_o_simbolos_falla_con_422`
tests/test_api_csv.py	Procesamiento de lotes (CU2)	• `test_cargar_csv_valido` • `test_cargar_csv_vacio_falla` • `test_cargar_archivo_no_csv_falla` • `test_csv_exportable_incluye_predicciones`
tests/test_api_metrics.py	Consulta y comparativa de modelos (CU5, CU6)	• `test_obtener_metricas_produccion` (verifica campos `precision_negativo`, `recall_negativo`, `f1_negativo`) • `test_comparar_modelos` (incluye la Línea Base mayoritaria inactiva)
tests/test_api_health.py	Monitoreo y estado del servicio (CU7)	• `test_consultar_salud_retorna_200_ok` (valida estado `"degraded"` sin modelo y `"ok"` con modelo)

tests/test_service_pipeline_integration.py	Integración segura con Scikit-Learn	• `test_pipeline_integracion_recibe_texto_crudo` (evita doble preprocesamiento) • `test_pipeline_nombre_preprocess_compatibilidad` • `test_palabras_clave_explicabilidad_modelo_real` • `test_estado_health_degraded_sin_modelo`

Resultado de la ejecución: La suite ejecuta **48 pruebas automáticas**, obteniendo un **100% de éxito (`48 passed`)** en un tiempo de ejecución inferior a 2 segundos.

5.5 Frontend

El frontend de **Opiniómetro** fue desarrollado como una Single Page Application (SPA) moderna utilizando **React**, **Vite** y **Tailwind CSS**. Su objetivo es proporcionar una interfaz limpia, reactiva y didáctica para la clasificación de opiniones en tiempo real y la visualización de la calidad analítica del sistema.

La interfaz se divide funcionalmente en dos vistas o módulos principales accesibles desde la barra superior de navegación:

```
[ OPINIÓMETRO ]
  ├── 1. Analizar reseña (Módulo de Inferencia y Pipeline NLP)
  └── 2. Métricas de modelos (Módulo de Rendimiento y Evaluación)
```

Pantalla 1: Analizar Reseña (`AnalizadorResenas.jsx`)

Esta pantalla permite al usuario ingresar una opinión en lenguaje natural y evaluar de inmediato su polaridad, desglosando la inferencia en tres bloques visuales:

1. **Entrada de Texto y Acción:**
   - Área de texto libre (`textarea`) donde el usuario escribe o pega la reseña (ej. *"la comida estuvo deliciosa"*).
   - Botón de acción **"Analizar Sentimiento"** con control de estado de carga (*loading*) y deshabilitación durante la llamada a la API (`POST /api/v1/predict`).
2. **Tarjeta de Resultado y Confianza (`ResultadoSentimiento.jsx`):**

- **Insignia de Sentimiento:** Etiqueta dinámica con código de color (verde para *Positivo*, rojo/ámbar para *Negativo*).
- **Metadatos de Inferencia:** Identificación del modelo en producción (*Regresión Logística TF-IDF*) y latencia de respuesta en milisegundos devuelta por el backend.
- **Medidor Circular de Confianza:** Gráfico radial SVG que muestra el porcentaje de certeza de la predicción.
- **Distribución de Probabilidad:** Barras comparativas que desglosan la probabilidad asignada a la clase positiva frente a la negativa.

3. **Recorrido Didáctico del Pipeline NLP (`NlpPasoAPaso.jsx`):**
   - Acordeón interactivo de 5 fases que visibiliza paso a paso la transformación del texto:
     1. **Original:** Texto crudo tal como fue redactado por el usuario.
     2. **Limpieza / Normalización:** Minúsculas, normalización de caracteres y remoción de signos no alfanuméricos.
     3. **Tokenización:** Segmentación del flujo de texto en unidades atómicas (tokens).
     4. **Lematización / Reducción:** Extracción de lemas o raíces léxicas y filtrado de *stopwords* preservando negaciones críticas.
     5. **Representación TF-IDF y Palabras Clave:** Listado y barras de impacto ponderado ($coeficiente \times TF - IDF$) de las palabras clave que determinaron el veredicto del modelo.

Pantalla 2: Métricas de Modelos (`MetricasModelos.jsx`)

Esta pantalla presenta el reporte de rendimiento global del modelo entrenado y la auditoría estadística del sistema, consumiendo los endpoints `/api/v1/metrics` y `/api/v1/metrics/compare`:

1. **Cuadrícula de Indicadores Globales (KPIs):**
   - **F1-Score Macro:** Métrica armónica equilibrada (ej. 92.82%).
   - **Precision Macro:** Exactitud global de predicciones positivas y negativas (ej. 95.35%).
   - **Recall Macro:** Cobertura de recuperación sobre ambas clases (ej. 90.58%).
   - **Precisión Negativa:** Precisión específica sobre la clase minoritaria crítica calculada como $\frac{TN}{TN+FN}$ (ej. 91.87%).
   - **Recall Negativo:** Tasa de verdaderos negativos recuperados calculada como $\frac{TN}{TN+FP}$ (ej. 81.63%).
   - **Muestras de Prueba:** Total de instancias evaluadas en el conjunto de test (ej. 16,143 reseñas).
2. **Matriz de Confusión Interactiva:**
   - Tabla de doble entrada formateada visualmente en cuadrantes según el resultado:

        - **Verdaderos Negativos (TN):** Reseñas negativas clasificadas correctamente (destacado en verde).
        - **Falsos Positivos (FP):** Errores donde una reseña negativa fue predicha positiva (destacado en advertencia/ámbar).
        - **Falsos Negativos (FN):** Reseñas positivas erróneamente marcadas como negativas.
        - **Verdaderos Positivos (TP):** Reseñas positivas clasificadas correctamente.
3. **Tabla Comparativa de Modelos:**
    - Vista tabular comparativa que contrasta el modelo en producción (*Regresión Logística*) contra las líneas base evaluadas (*Naive Bayes Multinomial*), comparando sus valores de *Accuracy*, *F1 Macro*, *Precision Macro* y *Recall Macro*, con distintivo visual para el modelo activo.

6. Uso de inteligencia artificial

En este proyecto se utilizaron asistentes de inteligencia artificial como apoyo, según lo permitido por las normas de la materia. Cada integrante declara a continuación para qué los utilizó en su parte. Todo el código generado o sugerido por IA fue revisado, ejecutado y verificado por la integrante que lo subió, quién puede explicarlo y modificarlo.

Sanchez Cuellar Melissa (Datos y NLP)

Herramienta utilizada: **Claude** (asistente de IA de Anthropic).

Se utilizó para:

- **Planificación:** analizar el documento del proyecto, comparar los temas sugeridos y organizar el trabajo por etapas.
- **Elección del dataset:** buscar datasets de reseñas en español y comparar alternativas.
- **Código:** como guía para escribir el script de descarga (data/descargar_datos.py), la función preparar_datos(), las mejoras del normalizador y del tokenizador, y las pruebas automatizadas.
- **Análisis exploratorio:** sugerencias de qué analizar y cómo graficarlo en los notebooks 01_eda.ipynb y 02_normalizacion_tokenizacion.ipynb.
- **Diagramas:** generación del diagrama de clases y del diagrama de casos de uso con PlantUML.
- **Revisión de código:** apoyo para revisar los pull requests de las compañeras antes de aprobarlos.
- **Redacción:** borradores de las secciones 1, 2, 3.1 y 3.2 de este informe.

Duran Marcelo Kelly Allison (Backend)

Herramienta utilizada: Antigravity / Gemini (asistente de IA de Google DeepMind).

Se utilizó para:

- **Diseño y arquitectura del backend:** estructurar la API REST desacoplada en 5 capas, definir la inyección de dependencias (`Depends`) y configurar los middlewares de CORS y ciclo de vida (*lifespan*) en FastAPI.
- **Preprocesamiento lingüístico (`FiltroStopWords`):** apoyo en la lógica de preservación de adverbios de negación (*"no"*, *"nunca"*, *"jamás"*, *"tampoco"*, *"sin"*) y conjunciones adversativas (*"pero"*), evitando la inversión involuntaria de polaridad en las reseñas.
- **Contratos de datos y validación defensiva:** redacción y calibración de los esquemas Pydantic (`api/schemas.py`) y sus validadores de campo para el rechazo de textos vacíos, longitudes no permitidas (mínimo 3 y máximo 5.000 caracteres) y filtrado de cadenas sin caracteres alfabéticos válidos.
- **Lógica del servicio e integración de modelos:** implementación del `ServicioPrediccion` y su modelo heurístico de respaldo resiliente (`ModeloHeuristicoRespaldo`); adaptación del servicio para recibir texto crudo eliminando el doble preprocesamiento y extracción dinámica del transformador entrenado desde `named_steps`.
- **Explicabilidad didáctica (`_palabras_clave`):** formulación del algoritmo que calcula el producto de coeficientes de regresión logística por pesos TF-IDF para extraer las palabras de mayor impacto (positivas y negativas) requeridas por el frontend.
- **Procesamiento por lotes CSV:** diseño de la detección automática de columnas de reseñas en archivos CSV y la generación del archivo con predicciones y probabilidades para descarga directa.
- **Pruebas automatizadas:** creación de la suite de pruebas unitarias y de integración con `pytest` y `TestClient` (`test_api_predict.py`, `test_api_csv.py`, `test_api_metrics.py`, `test_api_health.py` y `test_service_pipeline_integration.py`).
- **Redacción técnica:** elaboración de los borradores técnicos de las secciones 3.3 (Stop words), 5.1 (Capas del sistema), 5.2 (Diagrama de clases) y 5.4 (Endpoints de la API) de este informe.

7. Referencias

Dataset

1. **Rest-Mex 2025: *Researching on Evaluating Sentiment and Textual Instances Selection for Mexican Magical Towns*. IberLEF 2025, CIMAT. Disponible en: https://sites.google.com/cimat.mx/rest-mex-2025 (consultado el 31 de octubre de 2026).**
2. ***Overview of Rest-Mex at IberLEF 2025: Researching Sentiment Evaluation in Text for Mexican Magical Towns*. Disponible en: https://www.researchgate.net/publication/395956558**
3. **Hernández Guevara, L. A. *scala-spark-reviews* (copia del archivo `Rest-Mex_2025_train.csv`). GitHub. Disponible en:**

https://github.com/lahernandezguevara/scala-spark-reviews (consultado el 31 de octubre de 2026).

Librerías y herramientas

4. Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
5. Bird, S., Klein, E. y Loper, E. (2009). *Natural Language Processing with Python*. O'Reilly Media. (Librería NLTK, lista de stop words en español).
6. The pandas development team. *pandas: Python Data Analysis Library*. Disponible en: https://pandas.pydata.org
7. Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. *Computing in Science & Engineering*, 9(3), 90–95.
8. Python Software Foundation. *re — Regular expression operations*. Documentación de Python. Disponible en: https://docs.python.org/3/library/re.html
9. pytest. *pytest documentation*. Disponible en: https://docs.pytest.org
10. PlantUML. Disponible en: https://plantuml.com (usado para los diagramas de clases y casos de uso).
11. Ramírez, S. (2018). *FastAPI: High performance, easy to learn, fast to code, ready for production.* Disponible en: https://fastapi.tiangolo.com (consultado en noviembre de 2026).
12. Colvin, S. et al. (2017). *Pydantic: Data validation using Python type hints.* Disponible en: https://docs.pydantic.dev (consultado en noviembre de 2026).
13. Encode OSS. *Uvicorn: The lightning-fast ASGI server.* Disponible en: https://www.uvicorn.org (consultado en noviembre de 2026).
14. Joblib Development Team (2020). *Joblib: Running Python functions as pipeline jobs and lightweight object persistence.* Disponible en: https://joblib.readthedocs.io (consultado en noviembre de 2026).

Asistente de IA

11. Anthropic. *Claude* (asistente de inteligencia artificial). Disponible en: https://claude.ai (ver sección 6).
12. Google DeepMind (2026). *Google Antigravity / Gemini (asistente de inteligencia artificial para desarrollo de software).* Disponible en: https://deepmind.google